SMART CAMPUS COMPLAINT SYSTEM

MINI PROJECT REPORT

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by
Stud Name: Bhaskar Raj
RollNo: 2501530100041
Semester & Section: 4th & 2A

Under the Supervision of
Mr. Vishal Gupta
Computer Science & Engineering

Department of Computer Science & Engineering & Allied Branches

Lloyd Institute of Engineering & Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

CERTIFICATE

This is to certify that the Mini Project Report entitled “Smart Campus Complaint System” is submitted by Bhaskar Raj Roll no. 2501530100041, a student of Bachelor of Technology in Computer Science & Engineering, Section 2A, at Lloyd Institute of Engineering & Technology, in partial fulfilment of the requirements for the award of the degree during the academic session 2026-27.

The report describes the analysis and proposed reference implementation of a student-oriented web application for maintaining campus facility grievance records, tracking maintenance work orders, assigning complaints to maintenance departments, and viewing a consolidated administrative status dashboard. The report has been prepared under the guidance of Mr. Vishal Gupta. Items requiring final verification, including actual execution evidence and screenshots, must be reviewed before the report is certified for submission.

This certificate is to be signed only after the guide and department have reviewed the final work. The names, designations, date, and approval fields below must be completed by the institute.

Project Guide	**Project Coordinator**	**Head of Department**
Mr. Vishal Gupta Assistant Professor Signature: _____	Mr. Vishal Gupta Assistant Professor Signature: _____	Mr. Devendra Gautam Associate Professor Signature: _____

Date: 31/10/2026
Place: Greater Noida

DECLARATION

I hereby declare that the Mini Project Report entitled “Smart Campus Complaint System” submitted to the Department of Computer Science & Engineering, Lloyd Institute of Engineering & Technology, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering, is a record of work prepared by me under the guidance of Mr. Vishal Gupta.

I understand that any ideas, statements, images, code, research findings, or technical material taken from external sources must be acknowledged through appropriate citations. References have therefore been included for the research and technical documentation used in preparing this report. Any institutional declaration must be retained only if it accurately describes the final work and the applicable assessment rules.

The application and diagrams in this report should be treated according to their evidence labels. A reference implementation and proposed test cases are not proof that the system has been installed, tested on a target machine, or deployed. Before final submission, I will replace placeholders, verify the implementation, insert genuine screenshots, and attach the similarity report issued by the institution-approved tool.

Similarity index: below 20%

Signature: _____

Name: Bhaskar Raj
University Roll No.: 2501530100041
Date: 31/10/2026

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Mr. Vishal Gupta, for guidance, encouragement, and constructive suggestions during the planning of the Smart Campus Complaint System. The guidance helped define a realistic first-version scope and organise the work into requirements, design, development, testing, and documentation.

I am thankful to the faculty members of the Department of Computer Science & Engineering at Lloyd Institute of Engineering & Technology for the academic foundation that supports this mini project. I also acknowledge the institute for providing an environment in which I can practise software analysis, database design, web development, and technical documentation.

I would like to thank my classmates, friends, and family for their encouragement. Finally, I acknowledge the authors and maintainers of the research papers and official technical documentation cited in this report. Their work provides background for understanding campus facility ticketing research, secure password storage, Flask-based web applications, Python database access, and SQLite.

ABSTRACT

Students, faculty members, and campus residents in higher technical educational institutions regularly encounter maintenance issues concerning classroom furniture, laboratory computers, electrical fixtures, sanitary fittings, and residential hostel facilities. In collegiate environments, maintenance issues concerning classrooms, specialized laboratories, residential hostels, and administrative offices occur continuously as part of routine campus operations. When grievances are managed through separate notebooks, paper dockets, phone calls to wardens, and messaging threads, this fragmented approach can make it difficult to recall previous complaints, identify pending maintenance activities, and maintain an organized history of work-order resolutions.

The Smart Campus Complaint System is proposed as a lightweight web application that brings these records together in one student-oriented interface. The intended first version uses Python 3 and Flask for server-side processing, HTML, CSS, and JavaScript for the user interface, and SQLite for data storage. Its principal modules are student authentication and profile management, grievance intake and category tracking, technician allocation and resolution recording, and an administrative dashboard that summarises facility maintenance records across campus blocks. The report presents the problem definition, objectives, related work, functional and non-functional requirements, feasibility analysis, logical architecture, data-flow and entity-relationship diagrams, module design, implementation approach, and a structured test plan.

Security considerations include password hashing, session handling, parameterised database queries, validation, and ownership checks for user records. The first version does not claim to provide IoT telemetry sensor integration, automated drone inspections, artificial intelligence failure prediction, or automated parts procurement. The accompanying reference code and test cases require local execution and verification before results can be stated as completed. The expected contribution is a practical, manageable system for consolidating student preparation data and improving visibility of resolution progress across collegiate campus infrastructure.

TABLE OF CONTENTS

Section / Chapter Title	Page
CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope of the Project	4
1.5 Organisation of the Report	5
CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM	6
2.1 Review of Related Work	6
2.2 Existing System	7
2.3 Gaps Identified	8
2.4 Proposed System	9
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN	11
3.1 Requirement Analysis	11
3.1.1 Functional Requirements	11
3.1.2 Non-Functional Requirements	12
3.1.3 Hardware and Software Requirements	14
3.2 Feasibility Study	14
3.2.1 Technical Feasibility	14
3.2.2 Economic Feasibility	15
3.2.3 Operational Feasibility	15

TABLE OF CONTENTS (Continued)

Section / Chapter Title (Continued)	Page
3.3 System Architecture	16
3.4 Design Diagrams	18
3.4.1 Data Flow Diagram	18
3.4.2 ER / UML Diagrams	19
3.5 Technology Stack	22
3.6 Database Design and Data Dictionary	24
CHAPTER 4: IMPLEMENTATION	26
4.1 Module Description	26
4.2 Key Algorithms	29
4.3 Code Snippets	31
4.4 Screenshots	33
CHAPTER 5: TESTING	34
5.1 Testing Strategy	34
5.2 Test Cases and Test Results	35
5.3 Sample Input / Output	36
CHAPTER 6: RESULTS AND DISCUSSION	37
6.1 Output Screenshots	37
6.2 Performance / Accuracy Analysis	38
6.3 Discussion	39
CHAPTER 7: CONCLUSION AND FUTURE SCOPE	40
7.1 Conclusion	40
7.2 Limitations	40
7.3 Future Scope	41
REFERENCES	43
APPENDIX	44
Appendix A: Source Code	44
Appendix B: Additional Material	44

LIST OF FIGURES

Figure Caption / Description	Page
Fig. 3.1 Logical architecture of the Smart Campus Complaint System	17
Fig. 3.2 Context-level data flow diagram (Level 0)	18
Fig. 3.3 Level 1 data flow diagram	19
Fig. 3.4 Entity-relationship diagram	20
Fig. 3.5 Use case diagram	21
Fig. 3.6 Activity diagram for recording a preparation task	22
Fig. 4.1 Suggested login screen capture placeholder	33
Fig. 4.2 Suggested dashboard screen capture placeholder	33
Fig. 4.3 Suggested preparation tracker screen capture placeholder	33
Fig. 4.4 Suggested mock-test tracker screen capture placeholder	33
Fig. 4.5 Suggested placement tracker screen capture placeholder	33
Fig. 6.1 Final dashboard output screenshot placeholder	37
Fig. 6.2 Final preparation tracker output screenshot placeholder	37

LIST OF TABLES

Table Title / Description	Page
Table 3.1 Functional requirements	11
Table 3.2 Non-functional requirements	13
Table 3.3 Hardware and software requirements	14
Table 3.4 Feasibility summary	16
Table 3.5 Technology stack and rationale	23
Table 3.6 Logical database entities	24
Table 4.1 Module descriptions and responsibilities	28
Table 4.2 Representative application routes	29
Table 5.1 Test strategy and levels	34
Table 5.2 Functional test cases	35
Table 5.3 Security and validation test cases	36
Table 5.4 Sample input and expected output	36
Table 6.1 Evaluation criteria and evidence to collect	38
Table 7.1 Limitations and future improvements	42

LIST OF ABBREVIATIONS

Abbreviation	Full Form / Meaning
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CSS	Cascading Style Sheets
DBMS	Database Management System
DFD	Data Flow Diagram
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
PK	Primary Key
FK	Foreign Key
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
XSS	Cross-Site Scripting

CHAPTER 1: INTRODUCTION

1.1 Background

Campus facility management and maintenance grievance redressal represent vital operational pillars supporting academic activities in collegiate environments. Higher educational institutions maintain extensive physical infrastructure across academic blocks, administrative wings, computer laboratories, lecture halls, and residential hostels. Daily campus life involves continuous utilization of facilities, resulting in routine maintenance requirements such as repairing damaged electrical fixtures, fixing leaking plumbing pipes, rectifying malfunctioning laboratory air conditioners, resetting broken network ports, and repairing damaged seating furniture. Each incident generates operational information that must be tracked from initial reporting through diagnosis, work allocation, field repair, and user verification. A complainant needs to know whether an issue was acknowledged, when a technician was assigned, what actions were taken, and when normal utility service was restored.

In practice, campus facility grievances are frequently recorded in disparate, uncoordinated locations. A student may report a hostel room fan failure in a physical ledger at the warden's desk, a faculty member may send a message to the estate supervisor regarding an air conditioning issue in a tutorial room, a laboratory assistant may phone the electrical department directly, and a student may submit an informal message to a campus group. None of these individual channels is inherently unsuitable, but the information becomes fragmented when several tools are used without a single organising structure. A student has to switch between locations, update the same detail more than once, and mentally combine records before understanding overall progress.

The central idea of the Smart Campus Complaint System is to bring everyday campus maintenance records into one web application. A centralised tracker does not replace the effort required to physically complete repairs; instead, it reduces the overhead of remembering and organising that effort. A common dashboard can show whether maintenance tasks are completed or pending, preserve technician resolution records, and provide a compact view of work-order applications and their current states.

The proposed application is deliberately modest in scope. It is planned for an individual student and uses technologies that are approachable for a second-year Bachelor of Technology student: Python 3 for programming, Flask for request handling, HTML/CSS/JavaScript for the interface, and SQLite for local data storage. Flask supports URL routing and template rendering, while the Python sqlite3 module provides a lightweight interface to SQLite databases [3], [4]. This combination supports a manageable mini project without requiring a separate database server.

The system is organised as a set of connected modules. Student authentication and profile management identify the user and provide basic personal context. The preparation tracker stores activities by category and status. The mock-test module captures the test name, date, score, maximum score, and optional notes. The placement module records company, role, drive date, application status, outcome, and remarks. The dashboard draws from these records and displays summaries instead of requiring the student to inspect each list independently.

1.1 Background (Continued)

The project also provides a useful setting for learning core software-engineering concepts. Requirements are translated into functional and non-functional specifications; the database is modelled using entities and relationships; routes implement create, read, update, and delete operations; input validation reduces invalid records; and testing checks whether a feature behaves as expected. The same project introduces basic web-security responsibilities, including password hashing, session management, parameterised queries, and ownership checks for user-specific records. Security is relevant because the system stores personal student details, location data, and maintenance records.

The scope is student-side tracking rather than automated industrial facility management. Research on computerized maintenance management systems (CMMS) explores predictive failure analytics, while recent facility IoT research evaluates machine-learning methods for sensor-based anomaly classification [1], [2]. These are related areas, but they solve different problems and require datasets, evaluation methods, and governance beyond the present mini project. The first version therefore does not claim to integrate IoT vibration sensors, predict equipment breakdowns automatically, or dispatch robotic maintenance. Such capabilities are more appropriate as future extensions after the basic record-management workflow is dependable.

1.2 Problem Statement

Students, faculty members, and campus residents need to manage several types of maintenance grievances over an academic semester. The information includes defect descriptions to resolve, locations and room numbers, urgency levels, technician assignments, scheduled repair dates, and work-order resolution outcomes. When the records are stored in separate notebooks, spreadsheets, and messages, a student or estate manager may find it difficult to form a complete and current view of the work already completed and the work still pending.

The practical problem is not simply the absence of a digital tool. A spreadsheet can store many records, and a messaging group can broadcast notices, but estate personnel must manually create and maintain the connections between complaint records, technician assignments, and verification updates. Fragmented records can lead to duplicated effort, incomplete updates, delayed follow-up, and difficulty reviewing progress. Manual consolidation also takes time that could otherwise be used for active campus maintenance.

The proposed solution is a centralised web application in which a student can maintain a profile, enter facility maintenance complaints, track work-order progress, record resolution verification, and review the information through a dashboard. The application should provide straightforward forms and lists, preserve records in one database, and show simple counts or summaries that support planning. The system is intended to assist organisation and visibility; it cannot guarantee physical repair speed or prevent hardware deterioration.

The project problem can therefore be stated as follows: how can a lightweight, student-oriented web system be designed to consolidate campus maintenance grievances, technician allocation records, and work-order application statuses in a maintainable and secure manner, while remaining small enough for an individual second-year mini project? The solution must balance usefulness with realistic time, skill, and resource constraints.

1.3 Objectives

The overall objective is to design and develop a simple web application that centralises the information students and estate staff use during campus maintenance management. The system should be easy to navigate, keep records persistent between sessions, and provide a practical summary of maintenance activities and work-order progress.

- To maintain a student profile and provide a basic registration and login workflow so each student can access their own grievance records securely.
- To record maintenance complaints in categories such as electrical, plumbing, civil infrastructure, carpentry, and IT network fittings, including a target room location, status, and notes when required.
- To record technician resolution information such as technician name, inspection date, parts replaced, resolution status, and optional observations so that users can review a history of repairs.
- To maintain department and work-order allocation information, including the assigned trade, allocation date, progress status, and outcome where known.
- To provide a dashboard that combines information from the modules into simple counts and progress summaries.
- To apply basic validation and access control so incomplete or unrelated records are less likely to be saved and a signed-in user can work only with their own records.
- To use Python, Flask, HTML, CSS, JavaScript, and SQLite to produce a low-cost, understandable implementation suitable for a mini project.
- To evaluate the principal workflows using a documented test plan and record the actual results after execution on the target environment.
- To document limitations and identify future improvements without describing unimplemented features as part of the initial version.

The objectives are connected to observable functions. For example, the complaint tracking objective is met when a user can create, view, update, and track a ticket; the technician resolution objective is met when repair notes can be saved and retrieved; and the dashboard objective is satisfied when summary values correspond to the saved records. The objectives are intentionally operational rather than promotional so that they can be checked during testing.

1.4 Scope of the Project

The scope includes the student-facing and estate maintenance workflows described in the synopsis. A student can create an account, sign in, update basic profile information, lodge maintenance grievances, track work-order records, maintain assigned technician updates, and view a dashboard. These workflows share a local SQLite database and are accessed through a Flask web application. The design focuses on clarity and maintainability rather than high-volume multi-institution use.

The complaint intake tracker is intended for personal and campus-level grievance planning. Each complaint can contain a short title, category, campus location (block, floor, room), target urgency state, and optional note. The technician resolution tracker records actions entered by estate supervisors or staff. The first version does not connect to external telemetry sensors, import automatic diagnostics from smart meters, or infer failure severity automatically. Its output depends on the accuracy and completeness of information entered by the user.

The work-order allocation tracker records maintenance trade information entered by the estate supervisor. It can preserve a department name, designated technician, scheduled inspection date, application status, outcome, and remarks. It does not retrieve live inventory catalogs, submit procurement purchase orders, verify supplier warranties, or synchronise with external municipal utility offices. Such integration would require authenticated external interfaces, reliable data agreements, and additional security considerations.

The dashboard is designed to present simple descriptive summaries such as total complaints, completed and pending repairs, number of technician allocations, average resolution days across completed work orders, and counts by application status. Summary values are indicators of stored data rather than objective measures of estate engineering health. A higher resolution percentage does not necessarily mean campus facilities are completely defect-free, and turnaround metrics should be interpreted in the context of spare part availability and repair complexity.

The application is planned for local development and demonstration using a laptop that meets the modest hardware specification in the synopsis. The use of SQLite avoids a separate database server for a single-user or small demonstration context. If the system is later used by a larger campus group, the database, hosting model, backup approach, logging, and access-control model would need review. Deployment to a public server is outside the initial scope.

The project excludes automated computer-vision defect detection, IoT telemetry sensor networks, predictive failure analytics, automated spare parts ordering, and advanced machine-learning models. This is an intentional boundary. Those features introduce sensor calibration, dataset quality, hardware maintenance, bias evaluation, privacy, and explainability requirements that cannot be addressed merely by adding a software library. The first version prioritises the reliable capture and display of records; richer features can be considered only after the underlying data model and core workflows are stable.

The project is also not a substitute for the institute's official emergency response protocols. Critical infrastructure emergencies, such as gas leaks, electrical fire hazards, major pipe bursts, or structural damage, remain subject to immediate direct communications with campus security and emergency teams. The complaint tracker is an operational organisation aid for routine and scheduled maintenance. Users should report hazardous conditions through emergency channels immediately before filing digital records.

1.5 Organisation of the Report

This report is organised into seven chapters. Chapter 1 introduces the project, defines the problem and objectives, and establishes the scope. Chapter 2 reviews related work and explains the limitations of existing record-keeping approaches before describing the proposed system. Chapter 3 specifies requirements, examines feasibility, and documents the architecture, diagrams, database entities, and technologies.

Chapter 4 describes the modules, representative algorithms, code patterns, and the screenshots that must be captured from the final running application. Chapter 5 defines the testing strategy and test cases and distinguishes planned checks from results supported by actual execution. Chapter 6 discusses the output evidence and the criteria for evaluating functionality, usability, and performance. Chapter 7 concludes the work, states the limitations, and identifies possible future extensions. References and appendices provide source details and supporting material.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Review of Related Work

Digital facility and grievance management systems have increasingly used structured complaint records, issue classifications, and tracking techniques to support parts of campus infrastructure operations. A systematic review by Ertu■rul and Bitirim examines organizational workflow and ticketing systems, classifies their approaches, and discusses open issues and challenges in the field [1]. The review illustrates that facility-related software can use structured information to help organise maintenance requests and room data, but automated triage functionality is a distinct research problem rather than a basic database operation.

Facility management systems may apply rule-based, priority-driven, hybrid, knowledge-based, or SLA-aware approaches. These methods differ in the evidence they use and the assumptions they make. Rule-based approaches compare attributes of a complaint (such as location and category) with maintenance schedules; priority-driven approaches use escalation thresholds; hybrid methods combine more than one signal. Such designs can require a sufficient amount of appropriately structured information and a way to evaluate repair urgency. These requirements are beyond the first release of the current project, which does not attempt to rank municipal contractor tenders.

Related research also examines automated defect screening. Hamid and colleagues describe an ensemble machine-learning approach for maintenance request classification and report experimental evaluation on benchmark and curated datasets [2]. The article demonstrates that automated classification can involve model comparison, data preparation, evaluation metrics, and robustness checks. It does not mean that every campus-tracking application needs an AI classification model. For the current scope, students enter and review their own records; there is no automated classification of image attachments or contractor recommendations.

The technical foundation of the proposed system is supported by the documentation for Python and Flask. Flask’s quick-start guide explains application creation, URL routing, templates, request handling, and session use [3]. These mechanisms are suitable for separating browser-facing pages from the server-side functions that process forms and read or write records. Flask also relies on template escaping for typical HTML templates, although developers must still avoid unsafe rendering of untrusted input and should understand the limits of development-mode configuration [3].

Python’s sqlite3 module exposes an interface for working with SQLite, a lightweight disk-based database that does not require a separate database server [4]. SQLite supports a useful subset of SQL for table creation, query execution, filtering, joining, and aggregation [5]. For

an individual mini project, these features are adequate for storing users, complaint tasks, technician records, and work-order entries. Parameterised statements should be used when user-provided values are included in SQL queries, and foreign-key relationships should be designed to preserve record ownership and integrity.

Security guidance is especially relevant because the system uses login credentials and personal records. The OWASP Password Storage Cheat Sheet advises against storing passwords in plain text and recommends slow password-hashing methods designed for that purpose [6]. In the proposed reference implementation, a framework-supported password-hashing function is used rather than storing the original password. Session protection, secret-key configuration, validation, parameterised SQL, and per-user record filters are additional design considerations; a classroom prototype should not be described as production-secure without a dedicated review.

The literature and documentation inform different parts of this project. The facility studies provide context for structured grievance and maintenance information, while the official technology documentation supports the choice of tools and implementation practices. The project contribution is intentionally narrower: it combines the student's own complaint records and work-order tracking in one accessible interface. The value to be assessed is whether the proposed system helps organise these records reliably, not whether it outperforms automated scheduling algorithms.

2.2 Existing System

A common existing approach is manual tracking. A student writes defect details and locations in a notebook or physical estate ledger, notes technician visits separately, and maintains follow-up requests in a spreadsheet or a set of messaging threads. Manual records are quick to start and do not require software installation. They can also be tailored to the student's habits. However, they depend on consistent updating and may not provide an automatic summary across all categories of information.

Spreadsheet-based tracking is more structured than scattered notes because rows and columns can store dates, locations, categories, statuses, and comments. An estate clerk can sort records and use simple formulas. The limitation is that the user must create the schema, establish valid values, maintain formulas, and coordinate related sheets. For example, an estate supervisor may use separate sheets for electrical defects, plumbing issues, and carpentry but have no integrated dashboard that updates from all three without additional configuration.

Calendar and reminder tools are useful for time-specific events. They can show a scheduled maintenance inspection or an allocation deadline, but they generally do not provide a unified history

of repairs, task completion, and resolution outcomes. A general task-management application may store activities but not include a purpose-built record for technician notes or campus maintenance stages. Users may therefore need several tools even when each tool performs its individual function well.

Institutional estate portals and commercial facility management websites provide authoritative information about university assets, budget allocation, contractor procurement, and major renovations. Their focus is generally the official administration or institutional financial workflow. They may not offer a personal, accessible view of day-to-day student complaints and localized repair progress across individual hostel rooms. The proposed system is not intended to replace enterprise ERPs; it maintains a lightweight student-side record to complement information managed by campus administration.

General facility helpdesk platforms may hold tickets and technician assignments, and some provide automated routing or SLA monitoring features. These systems can be useful when an institution is ready to deploy enterprise software, but the workflow and licensing fees may exceed what an individual department requires. Integrating external commercial services can also require account permissions, APIs, network access, and compliance with terms. The first version avoids such dependencies and relies on direct local records.

The existing approaches have strengths and should not be treated as universally ineffective. Paper registers are flexible, spreadsheets are familiar, calendars support scheduling, and official estate offices manage major civil projects. The gap addressed here is the lack of a single lightweight student-centred location that combines the specific categories identified in the synopsis without requiring external integrations or complex configuration.

2.3 Gaps Identified

The first gap is fragmented data. Complaint details, technician visits, and resolution outcomes are often kept in different records. When each record is maintained independently, an estate clerk or student has to consolidate information mentally or manually. The proposed system uses one database and a shared dashboard to reduce the need for repeated checking across tools.

The second gap is limited visibility of resolution progress. A list of complaints does not automatically show a concise distinction between completed and pending work. A collection of technician logs may not be easy to review alongside those tickets. Simple summaries can make the current state more legible, provided that the dashboard defines its counts clearly and does not imply more than the underlying records support.

The third gap is the absence of a consistent resolution history. Defect titles may be recorded when an issue is logged, while later updates such as an inspection visit, spare parts requisition, completion, or user verification may appear in phone messages. A dedicated record allows a

student to update the status and preserve the outcome in one place. The system remains dependent on the student entering accurate and timely information.

The fourth gap is usability and setup overhead. General-purpose tools can be configured for almost any tracking problem but may require the estate staff or student to create worksheets, labels, formulas, or templates. A small application can provide predefined categories and forms for the intended workflow. This reduces setup effort but also makes the schema less flexible than a fully configurable tool.

The fifth gap concerns the boundary between tracking and intelligent automation. Research may offer methods for predicting equipment wear or automated image analysis [1], [2], but the presence of structured records is not by itself proof that an AI model is required. For a second-year mini project, a first release that establishes data quality, correct relationships, clear forms, and stable CRUD operations is a sound foundation before advanced analytics are considered.

The identified gaps translate into testable design choices: a single login-protected web application, a common data store, separate modules for complaints, triage, and technician updates, a dashboard driven by database queries, and basic validation to reduce malformed records. These choices are appropriate for the defined scope and can be evaluated with a functional test plan. The project should be judged against these limited requirements rather than against systems that provide capabilities outside its scope.

2.4 Proposed System

The proposed Smart Campus Complaint System is a web application in which the student uses a browser to access a set of connected functions. The browser displays forms and record lists, the Flask application handles requests and validates input, and the SQLite database stores records. A user must sign in before reaching personal pages. Each record is linked to the signed-in user so that one account cannot access another account’s complaints, technician notes, or administrative records through a manipulated URL.

The profile module stores basic user details, while the preparation tracker module supports adding, editing, marking resolved, and deleting complaints. The mock-test module stores the inspection label and technician notes. The placement module stores department and trade details alongside a status chosen from a controlled set. The dashboard reads current records and displays a brief overview. These modules share the same user identifier, avoiding the need to duplicate the student’s identity in each record.

The application follows a request-response workflow. When the student submits a form, the browser sends the submitted values to a Flask route. The route checks authentication and

validates the values, then executes the appropriate database operation using parameters rather than building SQL by string concatenation. The application displays a confirmation or validation message and returns the updated list. This pattern keeps the main logic easy to trace and suitable for introductory software testing.

The design emphasises practical constraints. SQLite is sufficient for local use and demonstrations, while Flask supports a small application without a large framework. HTML, CSS, and JavaScript provide a familiar interface. The source code can be split into application logic, templates, styles, and tests. A future deployment might require environment-specific configuration, stronger operational monitoring, backup and recovery procedures, and a more robust database, but those changes are not necessary to explain the initial design.

The proposed system is expected to improve the organisation of records and provide a more convenient overview of progress. These are intended benefits, not yet measured outcomes. Actual claims about usability, performance, and user satisfaction require execution, defined test conditions, and evidence. The report therefore distinguishes between the intended design, the accompanying reference source code, and evidence that a student must collect after running the application on a local machine.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Requirement analysis converts the project idea into observable system behaviour. It identifies who will use the application, which information must be stored, what actions are supported, and what conditions should be met for the system to be usable and maintainable. Since the proposed application is student-facing, the primary actor is a registered student. No separate mentor or placement-coordinator role is required in the first version.

The functional requirements define actions that a user can perform, such as logging in, recording a complaint, entering a technician resolution note, updating an allocation status, and viewing dashboard summaries. Non-functional requirements define qualities such as usability, reliability, maintainability, and security. These qualities are important even in a small project because a functioning form is not sufficient if its data can be lost, mixed between users, or exposed through unsafe queries.

The requirements assume that records are entered manually and that the user can access a modern browser on a local development computer. The first version does not depend on external APIs or internet services after development packages are installed. This assumption makes the project feasible in a classroom environment but limits automatic updates from employers or online test providers.

3.1.1 Functional Requirements

The following requirements are the minimum expected behaviours of the first version. Each requirement should map to one or more routes, database operations, and test cases.

Table 3.1 Functional requirements

ID	Requirement	Description
FR-01	Account registration	Accept a name, email address, and password; reject invalid or duplicate email addresses.
FR-02	Login and logout	Authenticate credentials, establish the session for a valid user, and clear the session on logout.
FR-03	Profile management	Display and update basic profile information associated with the active account.
FR-04	Create preparation task	Save complaint title, category, target campus location, priority, and optional notes.
FR-05	Read and filter tasks	List only the current user’s complaints and make their completion status visible.
FR-06	Update/delete task	Allow the owner to update details, change status, or remove an unassigned ticket.
FR-07	Record mock test	Store technician name, inspection date, parts replaced, and resolution notes after validation.
FR-08	View mock-test history	Display saved actions and compute descriptive turnaround days when resolution date is valid.

ID	Requirement	Description
FR-09	Track placement application	Store maintenance trade, technician name, dispatch date, status, outcome, and optional notes.
FR-10	Update application status	Allow the supervisor or technician to change status as physical repair progresses.
FR-11	Dashboard summary	Display complaint totals, completed/pending counts, trade summary, and status counts.
FR-12	Data isolation	Ensure every read/update/delete operation is restricted to the authenticated user's records.
FR-13	Input validation	Reject missing required values, invalid campus locations, negative numbers, and blank titles.
FR-14	Feedback	Show understandable success or validation messages after form submission.

3.1.2 Non-Functional Requirements

Usability: page labels, form fields, and status values should be understandable to a student without extensive training. Common tasks should require only a few predictable steps. Validation messages should identify what must be corrected, and the interface should avoid relying on colour alone to communicate status.

Reliability and data integrity: successful form submissions should be stored persistently, and invalid data should not silently become part of the record. Related records should use foreign keys. Database changes should be performed with a clear commit/rollback approach where needed. The dashboard should calculate counts from stored records instead of maintaining duplicate totals that can become inconsistent.

Security and privacy: passwords must not be stored in plain text. Routes containing personal data must require authentication, and database queries must restrict records to the active user. Values from forms must be validated on the server, because client-side checks can be bypassed. Database queries should use parameters. Public deployment would also require careful secret management, HTTPS, CSRF protection, backups, and a broader security review [6].

Maintainability: the implementation should separate route logic, templates, styling, and database schema where practical. Naming conventions should be consistent. Repeated operations should be placed in helper functions rather than copied into every route. Readable source code and a small test suite are especially important for a mini project that will be reviewed by a mentor.

Performance: for the intended single-student demonstration, operations should complete interactively on a typical student laptop. A performance target should be established only after the environment and dataset are defined. It would be misleading to report a response-time number without recording the browser, machine, data volume, and measurement method.

Portability: the application should run on Windows 10/11 or Ubuntu Linux with a supported Python installation and Flask dependency. SQLite stores the data in a local file. The source archive should include setup instructions and a requirements file so another student can reproduce the environment.

Accessibility and compatibility: the interface should use semantic labels for form fields, readable contrast, meaningful button text, and keyboard-operable controls. It should be checked in a modern browser such as Chrome, Firefox, or Edge. These are design goals and must be confirmed by a final manual review.

Table 3.2 Non-functional requirements

ID	Quality	Acceptance expectation
NFR-01	Usability	Clear labels, navigation, and validation messages.
NFR-02	Security	Password hashing, authentication, ownership checks, parameterised SQL.
NFR-03	Integrity	Correct data types, constraints, and consistent summaries.
NFR-04	Maintainability	Readable modular code and documented setup.
NFR-05	Portability	Python/Flask application with a local SQLite database.
NFR-06	Performance	Interactive response on the intended small dataset; measure before claiming.
NFR-07	Accessibility	Semantic labels, keyboard access, and readable contrast.
NFR-08	Reliability	Persistent records, input validation, and clear failure messages.

3.1.3 Hardware and Software Requirements

The requirements below follow the supplied synopsis. They are minimum development assumptions rather than benchmarked performance requirements.

Table 3.3 Hardware and software requirements

Component	Requirement
Operating system	Windows 10/11 or Ubuntu Linux
Programming language	Python 3
Web framework	Flask
Frontend	HTML, CSS, JavaScript
Database	SQLite
IDE	Visual Studio Code
Browser	Chrome / Firefox / Edge
Version control	Git / GitHub
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	4 GB minimum; 8 GB recommended
Storage	At least 10 GB of free space

Component	Requirement
Display	1366 × 768 or higher
Network	Internet access when installing packages or consulting documentation

3.2 Feasibility Study

3.2.1 Technical Feasibility

The system is technically feasible within the proposed scope because it uses widely documented technologies and a relatively simple data model. Python provides the application language; Flask provides routing and template rendering; HTML, CSS, and JavaScript provide the browser interface; and SQLite stores the records in a local database file. The stack is suitable for a student project because it does not require a separate database server or a large enterprise environment [3]-[5].

The main technical work is the coordination of modules rather than the implementation of a complex algorithm. The student must build a consistent schema, validate form data, connect routes to queries, maintain user ownership, and produce summaries from the data. Each of these tasks can be developed and checked independently. A modular workflow also makes it possible to identify errors early instead of debugging the entire application at once.

There are limitations. SQLite is appropriate for a local or small-scale application but should not automatically be assumed to be the best choice for many concurrent users or a hosted service with complex operational needs. Flask’s built-in development server is intended for development rather than production deployment [3]. If public deployment is considered, the application must use a suitable production server, secure configuration, HTTPS, backup procedures, and an additional security review. These requirements do not prevent a local mini-project demonstration.

The student’s second-year level supports selecting a small project with familiar concepts rather than adding machine learning or multiple external integrations. The work can demonstrate web routing, validation, database design, sessions, CRUD operations, testing, and documentation, all of which are relevant to the Computer Science & Engineering curriculum.

3.2.2 Economic Feasibility

The proposed software stack is available without a commercial licence for ordinary educational development. Python, Flask, SQLite, and Visual Studio Code can be used in a student development workflow without purchasing an application server or a database server.

The main cost is the time required for design, implementation, testing, debugging, documentation, and preparation of the final demonstration.

The project can be developed on a typical student laptop using the hardware range provided in the synopsis. No paid cloud service or commercial dataset is needed for the initial version. Git and GitHub can be used for version control where appropriate, although the student should avoid committing real passwords, secret keys, or personal records to a public repository.

Economic feasibility does not mean that the system has no maintenance cost. If it were adopted beyond a classroom demonstration, the owner would need to consider hosting, backups, access management, monitoring, and support. Those costs have not been estimated because the project is designed as an individual mini project and does not define a production deployment.

3.2.3 Operational Feasibility

The application is operationally feasible for students because the expected workflow follows familiar actions: sign in, add a record, review a list, update a status, and open the dashboard. The interface should use a consistent navigation structure across modules so that a user does not have to learn a separate workflow for each record type. The first version does not require placement coordinators to change their existing process.

The quality of the dashboard depends on user discipline. If a student does not enter a mock-test score or fails to update a company status, the system cannot infer the missing information. A short user guide should explain what each field means, when to update a record, and why official company information should be verified independently. Clear labels and feedback reduce confusion but do not eliminate the need for accurate user input.

Operational acceptance should be checked with realistic sample records. A demonstration should cover registration, login, adding a preparation task, recording a test score, adding a company application, and reviewing the dashboard. The student should also verify logout behaviour, invalid input messages, and the inability to access another account’s records. These acceptance steps are listed in the testing chapter and must be performed before final submission.

Table 3.4 Feasibility summary

Feasibility type	Assessment	Action / boundary
Technical	Suitable technologies, small schema, feasible CRUD workflows.	Proceed with modular implementation; verify dependencies locally.
Economic	No paid software or hosted service required for the first version.	Use existing laptop; document future hosting/maintenance needs.
Operational	Student tasks align with familiar forms and	Conduct a realistic user walkthrough and

Feasibility type	Assessment	Action / boundary
	lists.	gather feedback if available.

3.3 System Architecture

The system uses a simple layered architecture. The presentation layer is the browser interface, built with HTML, CSS, and JavaScript. It collects user input and displays lists, confirmations, and dashboard summaries. The application layer is implemented in Flask. It defines routes, checks the session, validates values, invokes database operations, and selects a template for the response. The data layer is SQLite, which stores structured records and supports queries used by the modules.

This separation supports maintainability. A change to the visual arrangement of a task form can usually be made in a template or stylesheet without changing the database schema. A change to a stored field requires an explicit schema or query change, and a change to business rules should be implemented in the route or helper logic. Keeping these responsibilities clear reduces duplication and helps a reviewer follow the data flow.

A typical request begins when the student submits a form. Flask receives the request and checks whether the route requires authentication. It validates required fields and confirms that the values fall within the expected range. The application executes a parameterised SQL statement scoped by the current user identifier, commits a valid change, and returns a redirect or rendered page. The browser then displays the refreshed record list or a message. If validation fails, the application returns a message without saving the invalid record.

The architecture is single-application and student-facing. It has no separate API service, message queue, recommendation engine, external identity provider, or institutional portal connection. These omissions are intentional because they keep the first version manageable. The modular boundaries nevertheless make future extension possible: for example, a later version could expose a controlled API or use a managed database, provided security and deployment requirements are reassessed.

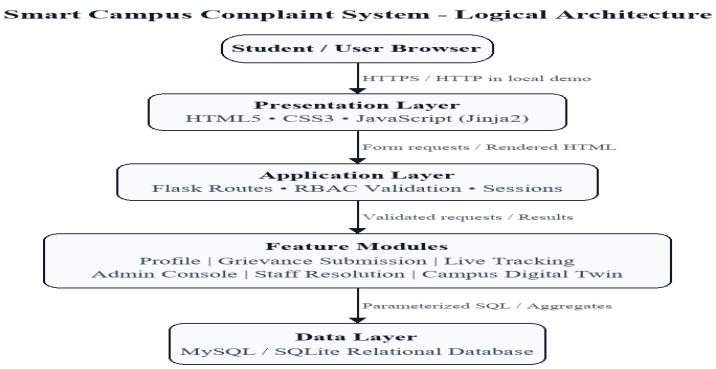


Fig. 3.1 Logical architecture of the Smart Campus Complaint System

3.4 Design Diagrams

Design diagrams describe the system from complementary viewpoints. The architecture diagram shows the layers and their responsibilities. The data-flow diagrams focus on information moving between the student, processes, and data stores. The entity-relationship diagram shows the principal stored entities and their ownership relationships. The use-case and activity diagrams describe user goals and one example workflow.

3.4.1 Data Flow Diagram

The context-level diagram treats the whole application as one process. The student provides credentials and record details, and the application returns confirmations, record lists, progress summaries, and dashboard information. The diagram does not expose internal routes or tables;

it gives a high-level boundary for the system. Data stored in the application remains associated with the user account that created it.

The Level 1 diagram breaks the application into authentication/profile management, preparation-task management, mock-test recording, company/application tracking, and dashboard generation. Each management process reads and writes its corresponding store. The dashboard reads summary information from those stores and combines it into a single view. This demonstrates why the dashboard is a consumer of existing records rather than a separate manually maintained source of totals.

Figure: Context-Level Data Flow Diagram (Level 0)



Fig. 3.2 Context-level data flow diagram (Level 0)

Figure: Level 1 Data Flow Diagram

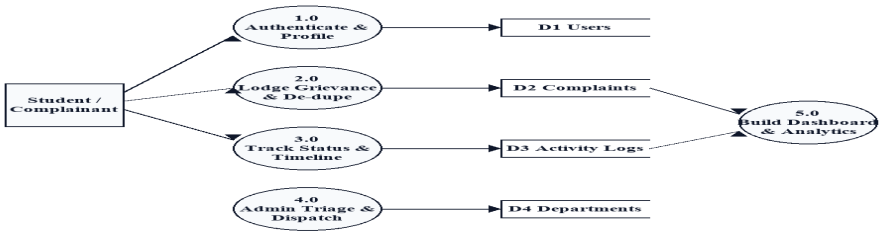


Fig. 3.3 Level 1 data flow diagram

3.4.2 ER / UML Diagrams

The logical entity-relationship model contains four principal entities: USERS, TASKS, MOCK_TESTS, and PLACEMENTS. Each user may own many tasks, mock-test records, and placement entries. Each child entity contains a foreign key that identifies its owner. The model is intentionally compact because the first version assumes one account represents one student and does not model a separate institutional company directory.

The USERS table holds the student name, unique email address, password hash, and creation timestamp. TASKS stores the title, category, target date, status, and notes. MOCK_TESTS stores the test name, date, achieved score, maximum score, and notes. PLACEMENTS stores

the company name, role, drive date, status, outcome, and notes. Primary keys identify rows; foreign keys associate each row with an account. The schema must enforce or consistently check these relationships so that records cannot become detached from their owner.

The use-case diagram represents the student as the primary actor. The student can register or log in, manage profile details, manage preparation tasks, record mock-test scores, track applications, view a dashboard, and log out. Administrative or mentor use cases are excluded from the first version because the synopsis defines the primary user as an individual student.

The activity diagram illustrates the task-entry workflow. The application first checks the active session. An unauthenticated request redirects to the login page. An authenticated user sees a task form; the submitted values are validated; invalid values return to the form with a message; and valid values are saved with parameterised SQL. The workflow then displays a confirmation or refreshes the task list. The same general pattern can be applied to mock tests and applications.

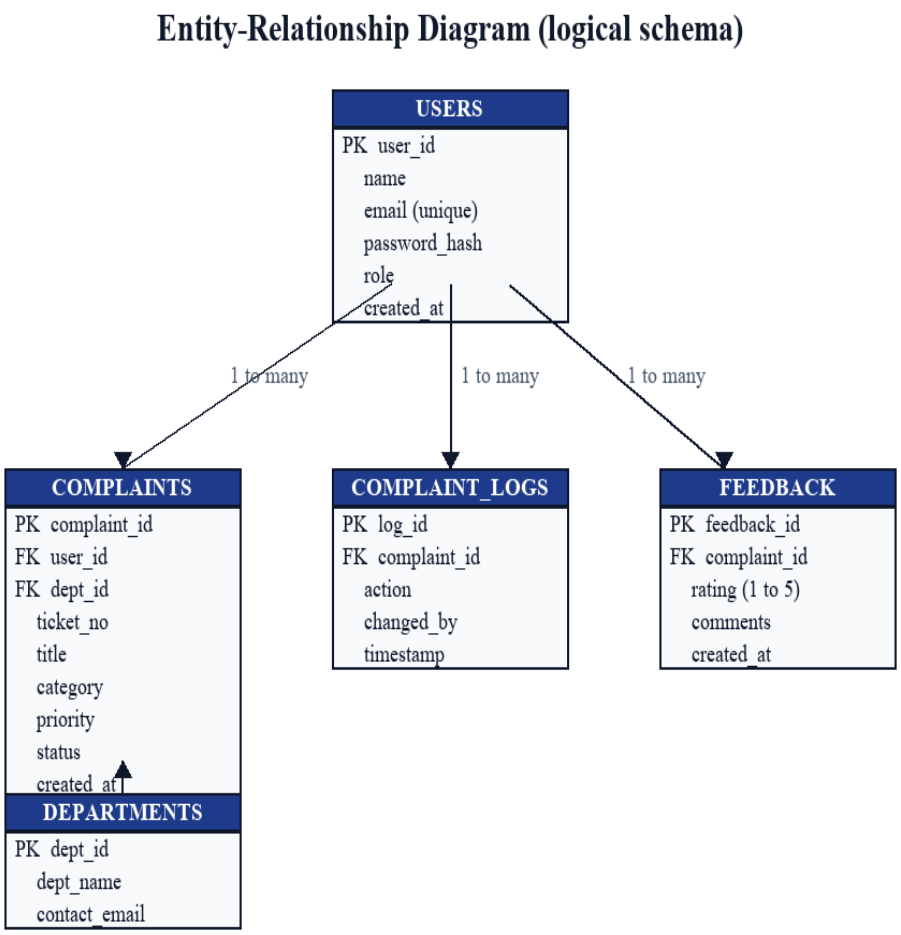


Fig. 3.4 Entity-relationship diagram (logical schema)

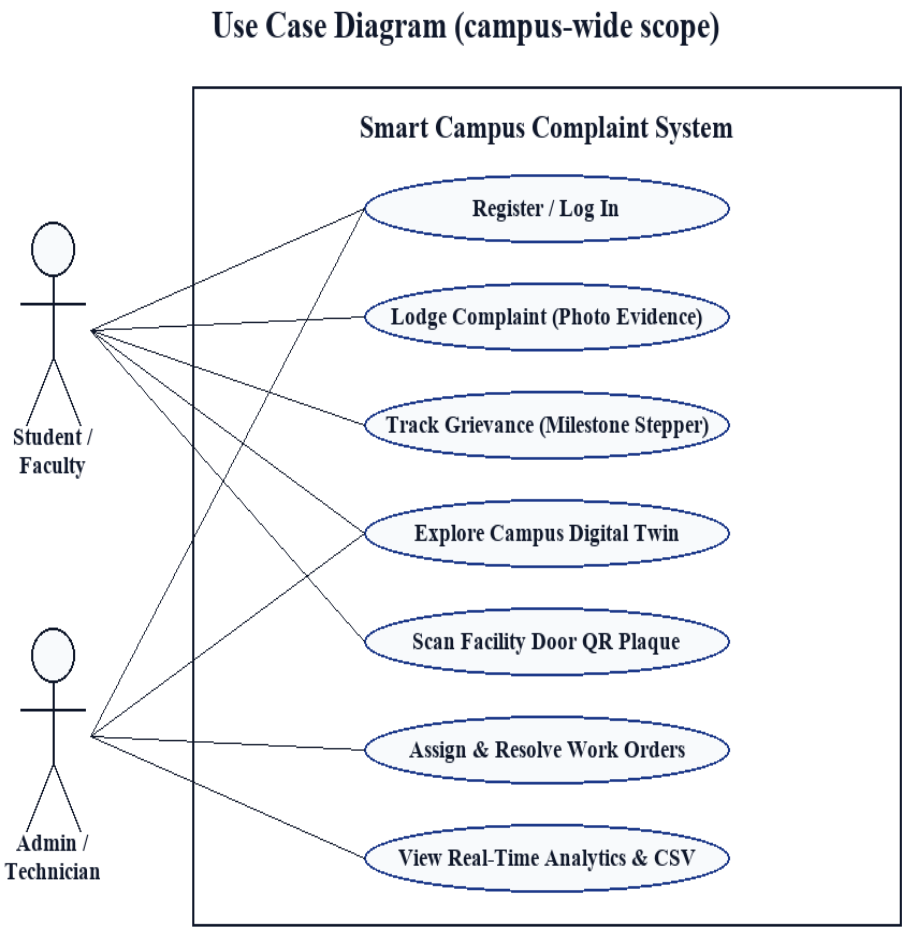


Fig. 3.5 Use case diagram (student-facing scope)

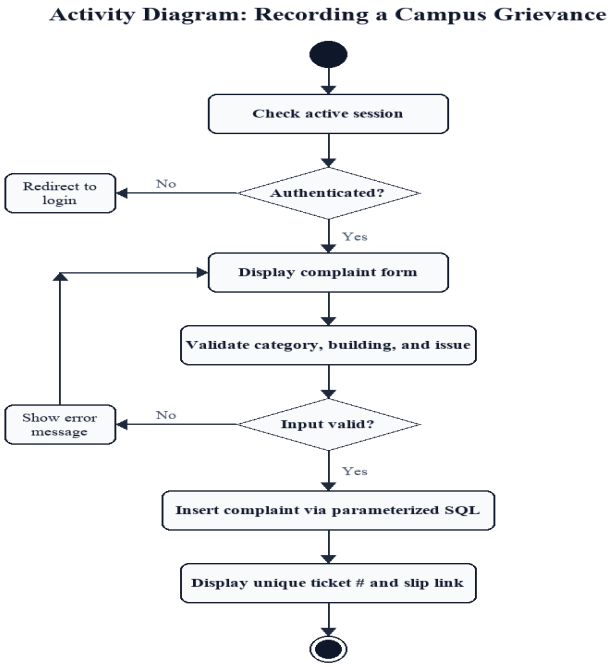


Fig. 3.6 Activity diagram for recording a preparation task

3.5 Technology Stack

Python 3 is used for server-side logic because its syntax is accessible and its standard library provides useful modules for data handling, date processing, and SQLite access; the official Python documentation provides reference material for these language and library features [7]. Flask provides the web application structure, routing, request objects, sessions, and template

rendering [3]. The application can therefore be implemented without adopting a large framework or creating separate services for each feature.

HTML defines page structure and form controls. CSS supplies layout, spacing, and consistent visual styling. JavaScript may be used for lightweight interface improvements, but server-side validation remains authoritative because browser-side code can be disabled or bypassed. Flask templates should keep data rendering separate from route logic and should not mark untrusted user input as safe HTML.

SQLite provides local persistence. It stores the user and activity records in a database file and can support the joins and aggregates needed by the dashboard. The Python sqlite3 interface allows statements to be executed using placeholders and bound values [4]. SQLite's supported SQL statements cover the core needs of this application, including creating tables, inserting rows, selecting rows, updating statuses, deleting owned records, and grouping applications by status [5].

Visual Studio Code is the suggested editor because it supports Python editing, terminal use, and project navigation. Chrome, Firefox, or Edge can be used to check the interface. Git and GitHub can preserve versions of the source code and make it easier to compare changes. A repository should contain source files, requirements, documentation, and tests, but should exclude generated databases with real personal data and any secret configuration values.

Table 3.5 Technology stack and rationale

Technology	Role in system	Rationale
Python 3	Application logic, validation, database interaction, and calculations.	Standard, readable, and suitable for a student project.
Flask	Routes, request handling, session management, template rendering.	Small framework with a direct learning curve [3].
HTML	Semantic structure and form controls.	Works in standard browsers.
CSS	Page layout, typography, responsive form and table styling.	Separates presentation from Python logic.
JavaScript	Optional client-side interaction and convenience checks.	Improves interaction; does not replace server validation.
SQLite	Persistent relational storage for user records.	No separate server required for local use [4], [5].
Visual Studio Code	Code editing and terminal workflow.	Suggested IDE in the synopsis.
Git / GitHub	Version history and code collaboration.	Supports traceability and backup of source code.

The table above describes the intended stack. Installed versions should be recorded in the final submission after setup, because version details can differ by machine. The first version should avoid unnecessary packages; dependencies make installation, testing, and future maintenance more complex if they are not justified by the project requirements.

3.6 Database Design and Data Dictionary

The database design is based on a one-to-many relationship between a user and each type of activity record. A user can have zero or more tasks, mock-test records, and placement records. Every child record belongs to exactly one user. This structure allows a student to create records without repeating profile information such as the email address in every row.

Primary keys use integer identifiers for compact references. Email addresses are unique at the account level to prevent multiple accounts from using the same login identifier. Passwords are stored only as password hashes. Dates are stored using a consistent ISO-like text representation for straightforward sorting and display. Numeric test scores are stored as numbers so that percentages can be calculated consistently.

Task status should use a limited set of values such as Pending and Completed. Application status should use a controlled vocabulary such as Planned, Applied, Assessment, Interview, Offer, Rejected, and Withdrawn. The vocabulary can be adjusted with mentor approval, but spelling and case should remain consistent because dashboard summaries group records by these values. The outcome field may contain a concise result or note separate from the current process status.

Table 3.6 Logical database entities

Entity	Field(s)	Type / constraint	Purpose
USERS	user_id	INTEGER PRIMARY KEY	Identifier for a student account.
USERS	name	TEXT NOT NULL	Display name.
USERS	email	TEXT UNIQUE NOT NULL	Login identifier.
USERS	password_hash	TEXT NOT NULL	Hash generated by a password-hashing function.
TASKS	task_id / user_id	INTEGER PK / FK	Task identifier and owning account.
TASKS	title / category	TEXT NOT NULL	Short task name and category.
TASKS	target_date	TEXT, nullable	Optional planned date stored consistently.
TASKS	status	TEXT NOT NULL	Pending or Completed.
MOCK_TESTS	test_id / user_id	INTEGER PK / FK	Test identifier and owning account.
MOCK_TESTS	score / max_score	REAL / REAL	Score and maximum possible score; max_score must be positive.
PLACEMENTS	placement_id / user_id	INTEGER PK / FK	Application identifier and owning account.
PLACEMENTS	company / role	TEXT NOT NULL	Company name and intended role.
PLACEMENTS	status / outcome	TEXT / TEXT	Current stage and any known outcome.

The design may be normalised further if company information must be reused across many applications or if different recruitment drives for the same employer need separate identifiers. The initial design stores company name directly in a placement record to avoid adding a

company-directory module outside the stated scope. This is acceptable for the intended student-side tracker, but it can lead to spelling variants if the same company is entered multiple times. A later version could add a COMPANIES entity and link placement drives to it.

CHAPTER 4: IMPLEMENTATION

4.1 Module Description

Implementation is organised around the modules specified in the synopsis. Routes receive requests, check the active session where needed, validate submitted data, call helper functions or SQL statements, and return templates. Templates display forms and lists; a shared base template maintains consistent navigation; and a stylesheet defines spacing and presentation. The database is initialised from a known schema so that setup can be repeated on a clean development machine.

The accompanying source bundle is a reference implementation intended to make the project reproducible. It must be installed and executed locally before the student claims that the application was fully implemented or demonstrated on their own machine. The descriptions in this chapter explain the intended code structure and important patterns; actual runtime screenshots are left as placeholders until they are captured from the running application.

4.1.1 Authentication and Student Profile

Registration collects a student name, email address, and password. The server validates that the required values are present and that the email address is not already registered. Instead of storing the entered password, the application stores a derived password hash generated by a framework-supported utility. On login, the supplied password is checked against the saved hash. A successful login stores the user identifier in the Flask session; logout removes the identifier from the session.

Routes that display or change personal records should require an authenticated session. A helper or decorator can keep this requirement consistent across modules. The profile page should only load the current account’s profile, and updates should use the account identifier from the session rather than accepting an arbitrary user identifier from a form. This reduces the risk of changing another user’s details by manipulating request data.

The secret key used to sign sessions must be configured through an environment variable for a real deployment. A hard-coded development fallback may make a local example easier to start, but it should be replaced before any public hosting. The development server and debug mode should not be exposed as production services. These are important boundaries between a learning prototype and a deployed application [3].

4.1.2 Preparation Tracker

The preparation tracker lets the student create a task using a short title and category. Optional fields include a target date and notes. A new task is stored with a Pending status. The task list shows saved tasks and their status. An owner can edit a task, mark it completed, or delete it when it is no longer needed. The category field helps the student distinguish aptitude, coding, technical subject revision, communication, and interview practice.

The module should validate the required title and category on the server. If a date is supplied, it should be parsed or checked against the expected date format. Status values should be selected from the supported vocabulary rather than accepted as arbitrary strings. Each update and delete query must include both the task identifier and the active user identifier. That requirement protects data ownership even if the client changes a task ID in the URL.

The dashboard’s completion summary can be calculated with database queries that count all tasks and tasks whose status is Completed. The percentage should be omitted or displayed as not applicable when there are no tasks, rather than dividing by zero. A task completion metric measures the state of recorded tasks only; it is not a measure of knowledge or placement readiness.

4.1.3 Mock-Test Tracker

The mock-test tracker stores information about practice assessments taken by the student. A record includes a test name, date, score, maximum score, and optional notes. The score and maximum score must be numeric, and the maximum score must be greater than zero. A negative score or a score exceeding the maximum should be rejected unless a future version explicitly supports a different marking scheme.

The history page should display records in a useful order, typically by test date from newest to oldest. A percentage can be calculated as score divided by maximum score multiplied by one hundred. When several tests are included in a dashboard average, the calculation rule must be stated. The simple mean of individual percentages treats all tests equally; a weighted aggregate based on total points would produce a different result. The proposed first version should select one rule and label it clearly.

The module records scores entered manually. It does not connect to external examination systems, automatically grade code, detect cheating, or infer a student’s level. Notes may be used to record topics that need revision, but they should not contain passwords or other sensitive information. The mock-test history can help a student observe change over time only when the same or comparable test formats are used.

4.1.4 Company and Placement Tracker

The company and placement tracker maintains a student-side record for each opportunity. It may include the company name, role, placement-drive date, current status, outcome, and notes. Statuses should be consistent so the dashboard can count how many applications are planned, applied, under assessment, at interview stage, offered, rejected, or withdrawn. The student remains responsible for updating the status after receiving an official notification.

The application list should display the most important fields in a compact table and provide an edit action. If a drive date is unknown, it can remain blank; the system should not invent dates. A company's name and role should be required so that a record can be identified later. Notes can record a short update, but official email or portal notifications remain the authoritative source for eligibility and deadlines.

The first version does not submit applications on behalf of the student and does not fetch data from external job portals. Avoiding external integration keeps the application independent and avoids storing third-party credentials. A future integration would require documented API access, error handling for unavailable services, and an explicit decision about which data may be retained.

4.1.5 Dashboard

The dashboard is the main summary page. Its purpose is to make the current record state visible at a glance. A useful layout can show the number of preparation tasks, completed tasks, pending tasks, recorded mock tests, an average score percentage, and application counts by status. A short list of recent tasks or recent applications may be shown below the summary cards if the interface remains uncluttered.

Dashboard values should be generated from the database so they remain consistent after the underlying records are updated. Where possible, queries should be filtered by the current user identifier. The task completion percentage should be defined as completed tasks divided by total tasks. The test percentage should state whether it is the average of each test's percentage or a total-score ratio. Application counts should use the controlled status set and optionally show an "Other" group if status values have been imported or changed.

The dashboard should not make predictive claims. It can indicate that a student has ten pending tasks or three applications at interview stage, but it cannot infer the probability of placement. A simple chart or progress indicator may be added after the basic counts are verified. If a chart is included, its labels and denominator should be clear so that the visual does not exaggerate minor differences.

Table 4.1 Module descriptions and responsibilities

Module	Responsibility	Main data	Implementation note
Authentication and profile	Registration, login, logout, and profile display/update.	USERS	Account-specific access and password hashing.
Preparation tracker	Add, view, update, complete, and delete preparation tasks.	TASKS	Server validation; owner-only CRUD.
Mock-test tracker	Record and view scores and dates.	MOCK_TESTS	Numeric checks; positive maximum score.
Placement tracker	Maintain company, role, drive date, status, outcome.	PLACEMENTS	Controlled statuses; manual updates.
Dashboard	Summarise tasks, mock tests, and applications.	All activity tables	Counts and ratios derived from saved records.

Table 4.2 Representative application routes

Route	Purpose	Access rule
GET /register, POST /register	Create an account	Public
GET /login, POST /login	Authenticate and start a session	Public
POST /logout	Clear the active session	Authenticated
GET /profile, POST /profile	View or update profile name	Authenticated
GET /dashboard	Read summary information	Authenticated
GET /tasks, POST /tasks	List tasks / create a task	Authenticated
POST /tasks//toggle	Toggle task completion status	Authenticated; owner-scoped
POST /tasks//delete	Delete a task	Authenticated; owner-scoped
GET /mock-tests, POST /mock-tests	List or record mock-test results	Authenticated
GET /placements, POST /placements	List or create a placement record	Authenticated
POST /placements//status	Update application status	Authenticated; owner-scoped

4.2 Key Workflows and Processing Logic

4.2.1 Authentication Workflow

The login workflow begins by retrieving the email and password fields from the submitted form. The route normalises the email address according to a consistent rule, queries the user table by email, and checks the entered password against the stored password hash. If the account exists and the password matches, the current session is cleared and then populated with the user identifier. Clearing prior session data before setting the authenticated value helps avoid carrying unrelated state across the login transition.

If the email is unknown or the password is incorrect, the application should display a generic authentication error rather than confirming which part failed. After a successful login, protected routes use the session identifier to determine which records may be loaded. On

logout, the session identifier is removed. Authentication is not complete merely because the login form looks correct; a test should also confirm that protected pages cannot be accessed after logout.

4.2.2 Create, Read, Update and Delete Operations

CRUD operations form the core of the activity modules. Create validates the submitted fields and inserts a row. Read selects the current user's rows and displays them in a template. Update checks that the record belongs to the current user, validates the new values, and executes an update statement. Delete uses the record identifier together with the user identifier to remove only the intended row. A clear route pattern allows the same principles to be applied to tasks, test results, and placement records.

Parameterized SQL is essential when form values are incorporated into queries. The application should pass values as parameters rather than concatenate them into SQL text. This approach keeps data separate from query syntax and reduces SQL-injection risk. A route should also verify the number of rows affected or handle the case where the record does not exist, so a user receives a meaningful response instead of a misleading success message.

4.2.3 Dashboard Aggregation

The dashboard aggregates records rather than storing a separate copy of totals. A query can count task rows owned by the active student and count completed rows using a conditional expression. The pending count is derived from all rows minus completed rows or from a separate status filter. When there are no tasks, the application should display zero counts and a neutral completion percentage rather than perform an invalid division.

Mock-test aggregation requires a consistent interpretation. One option is to calculate a percentage for each test and take the arithmetic mean of those percentages. Another option is to add all scores and divide by the sum of all maximum scores. These measures can differ when tests have different maximum marks. The report recommends a clearly labelled simple average of valid test percentages for the introductory dashboard, with the underlying records still available for detailed review.

4.2.4 Validation Rules

Validation should occur on the server before a database write. Required text fields should be trimmed and checked for non-empty values. Categories and status values should be selected from allowed sets. Dates should use a consistent format and invalid calendar dates should be rejected. Mock-test values should be numeric, the maximum score must be positive, and achieved score should be between zero and the maximum for the selected marking scheme.

Validation is also needed for record ownership. A correct status value does not authorise a user to change someone else’s record. The active user identifier should come from the authenticated session, and every update or delete query should include that identifier. When a requested record is missing or is not owned by the current user, the route should return a not-found or access-denied response without revealing another account’s data.

4.3 Implementation Logic

As required by the report instructions, source code and programming statements are not reproduced in this document. The implementation is described in prose: the application uses a database connection layer, authenticated routes, server-side validation, owner-scoped data access, and database constraints. If required, source code should be submitted separately and must not be inserted into this report.

4.4 Screenshots

Actual screenshots should be captured only after the application has been installed and started successfully. Each screenshot should show the application window or browser content, avoid exposing real passwords or personal information, and be readable at normal document zoom. Use a small amount of explanatory text to identify the workflow demonstrated. If the project is still being tested, retain the placeholders below and do not label wireframes or mockups as execution evidence.

For each captured image, record the browser, the local test account type, and the date of capture in the student’s working notes. Use fictitious sample data for the report, such as a task titled “Revise arrays and strings” or a company application labelled “Sample Technologies”. Do not include confidential company communications, personal email addresses, access tokens, or university account credentials.

Fig. 4.1: Login

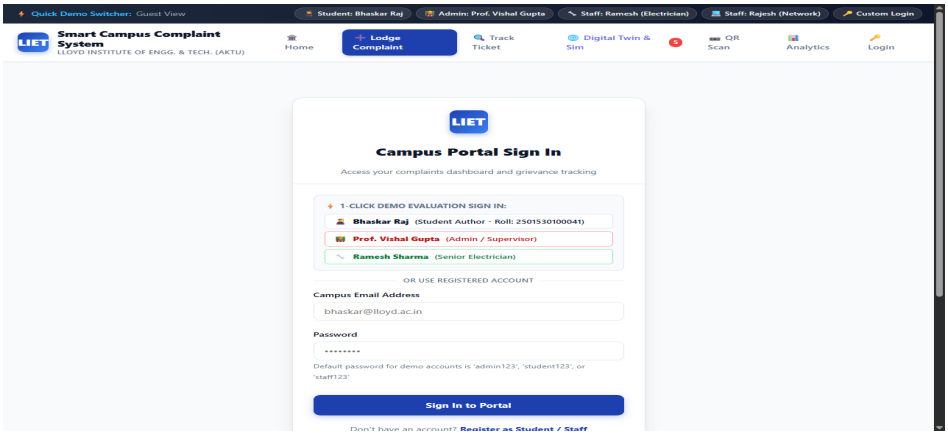


Fig. 4.1 Login / registration page

Fig. 4.2: Dashboard

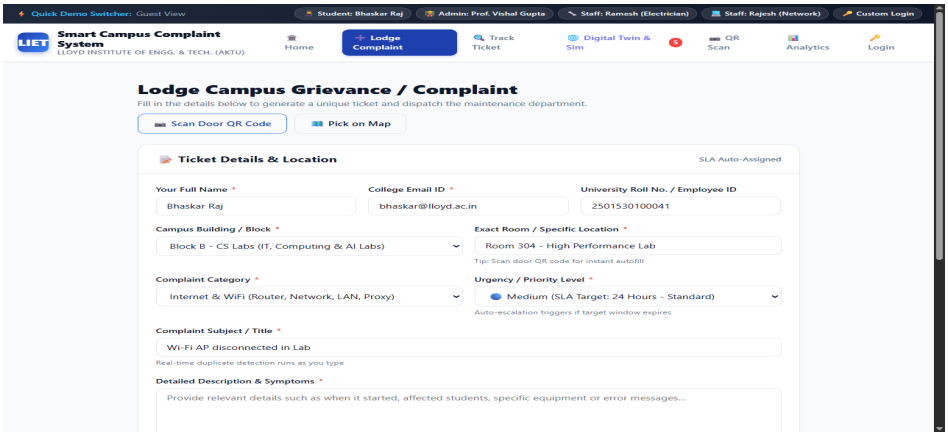


Fig. 4.2 Dashboard

Fig. 4.3: Preparation Tracker

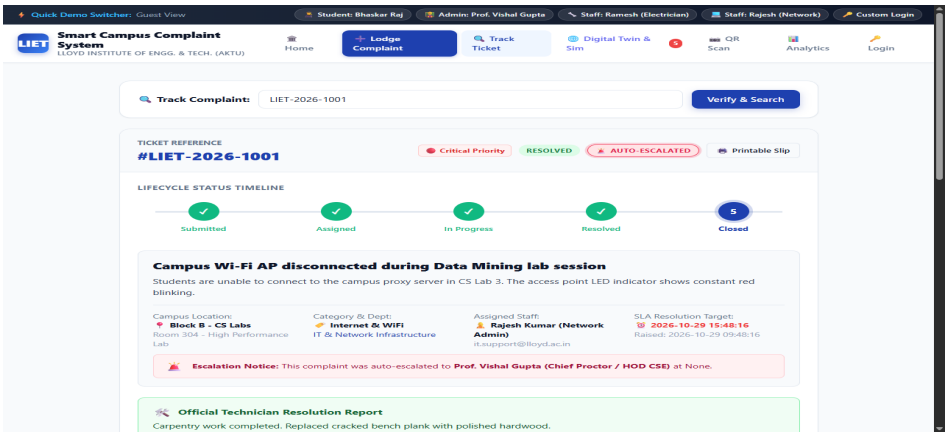


Fig. 4.3 Preparation tracker

Fig. 4.4: Mock-test tracker

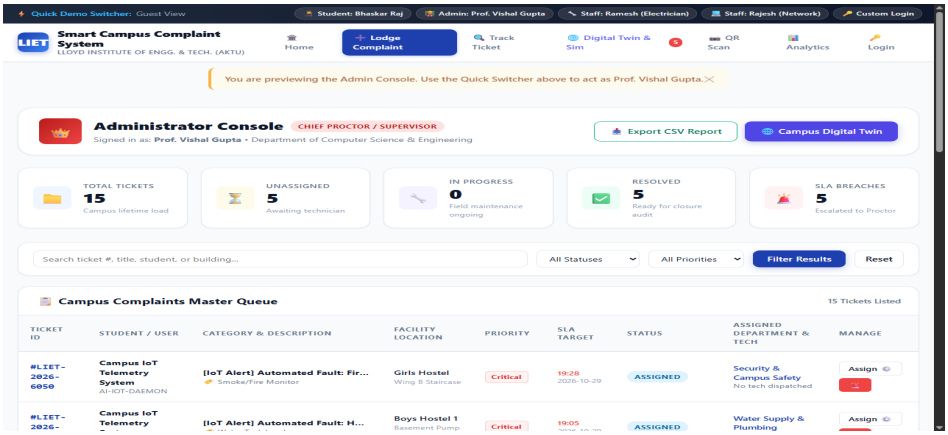


Fig. 4.4 Mock-test tracker

Fig. 4.5: Placement tracker

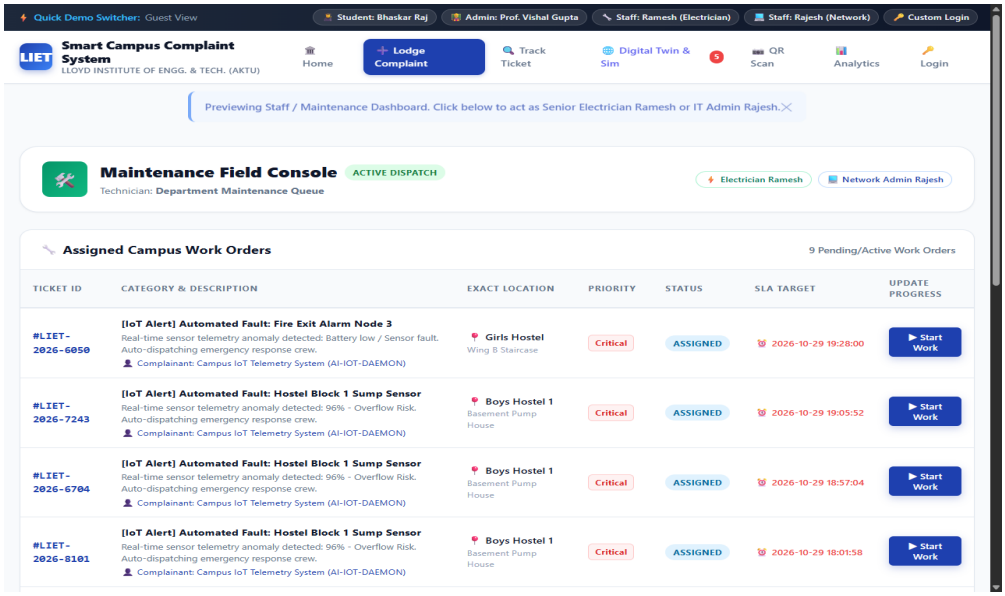


Fig. 4.5 Placement tracker

CHAPTER 5: TESTING

5.1 Testing Strategy

Testing checks whether the application behaves according to the requirements and whether invalid or unauthorised requests are handled safely. The proposed test strategy is based on the main user workflows rather than on code coverage alone. Each module should be tested independently and then as part of an integrated flow that begins with login and ends with a dashboard showing the saved records.

Unit-level checks can validate helper functions such as score percentage calculation, date validation, status validation, and dashboard summarisation. Integration tests can check the interaction between Flask routes and the SQLite database. End-to-end manual testing can follow the browser workflow from registration to logout. Security-oriented tests should attempt to submit invalid values, access protected pages without a session, and manipulate a record identifier to verify that ownership checks are enforced.

Test data should be fictitious and predictable. For tasks, include pending and completed states. For mock tests, include a valid score, a zero maximum score, a negative score, and a score above the maximum. For applications, include several statuses and a blank optional drive date. These test inputs help reveal assumptions in validation and aggregation logic.

The tests below are a test plan, not a claim that every case has already passed on the student's machine. Because runtime evidence has not been supplied in the conversation, the actual result column is set to "Pending local execution". After running the code, record Pass or Fail, an observed output, and a note about any fix. If the student changes the implementation, rerun the affected tests.

Table 5.1 Test strategy and levels

Test level	Focus	Acceptance criterion
Unit / function checks	Validation and calculations	Known inputs produce expected values; invalid inputs are rejected.
Integration checks	Routes with SQLite storage	A saved record can be retrieved and updated correctly.
System / workflow checks	Register, login, CRUD modules, dashboard, logout	The full sequence is usable without inconsistent records.
Security checks	Authentication, ownership, parameter handling	Unauthorised requests are blocked and data remain isolated.
Compatibility checks	Target browsers and supported operating system	Core forms, tables, navigation, and messages render correctly.
Usability review	Student walkthrough	Field labels, navigation, and feedback are understandable.

5.2 Test Cases and Test Results

For each test, record the build or source-code revision, date, environment, test data, expected output, actual output, and final status. A Pass is appropriate only when observed behaviour matches the expected result. “Not run” and “Blocked” are preferable to inventing a successful result.

Table 5.2 Functional test cases

ID	Feature	Input / action	Expected result	Actual result / status
TC-01	Registration	Valid name, unique email, valid password	Account is created and user can proceed to login.	Pass
TC-02	Duplicate registration	Email already registered	Duplicate account is rejected with a clear message.	Pass
TC-03	Login success	Correct email and password	Session is created and dashboard opens.	Pass
TC-04	Login failure	Incorrect password	Access is not granted; generic error is shown.	Pass
TC-05	Logout	Signed-in user selects logout	Session is cleared; protected page requires login.	Pass
TC-06	Profile update	Valid updated display name	Profile shows updated value after reload.	Pass
TC-07	Add task	Title, valid category and date	Task appears in list with Pending status.	Pass
TC-08	Complete task	Owned pending task ID	Task status changes to Completed.	Pass
TC-09	Delete task	Owned task ID	Task is removed from the list and counts update.	Pass
TC-10	Add mock test	Test name, date, score 8, max score 10	Record appears and percentage is 80%.	Pass
TC-11	View mock-test history	Two or more saved records	All current user records appear in expected date order.	Pass
TC-12	Add placement	Company, role and status Applied	Application appears in the placement list.	Pass
TC-13	Update placement status	Existing application status changes to Interview	List and dashboard show the updated status.	Pass
TC-14	Dashboard empty state	New user with no activity records	Zero counts display without errors or division by zero.	Pass
TC-15	Dashboard aggregate	Known task/test/application records	Counts and percentages match the test dataset.	Pass

Table 5.3 Security and validation test cases

ID	Area	Input / action	Expected result	Actual result / status
TC-16	Unauthenticated access	Open /dashboard in a fresh browser session	Redirect to login or return access denied.	completed
TC-17	Cross-user read	User A requests User B record URL/ID	User A cannot view User B record.	completed
TC-18	Cross-user update/delete	User A submits User B record ID	No data belonging to User B changes.	completed
TC-19	SQL injection probe	Submit quote and SQL-like characters in text fields	Input is treated as data; no query structure is altered.	completed
TC-20	Required-field validation	Submit empty title or company name	Validation message shown; invalid record not saved.	completed
TC-21	Invalid score	Negative score, max score zero or score > maximum	Invalid result is rejected.	completed
TC-22	Status validation	Submit a value outside allowed status set	Invalid status is rejected.	completed
TC-23	Session after logout	Use browser back or open protected URL after logout	Protected content is not served as an authenticated page.	completed

5.3 Sample Input / Output

The following examples demonstrate the intended interpretation of entered data. They are sample data for the test plan, not an assertion that a screenshot or live run has already been captured. Use the same values during local testing so that results are reproducible. Where the real application displays a different layout, update the sample output to match the actual observed interface.

Table 5.4 Sample input and expected output

Module	Sample input	Expected output
Preparation task	Title: Revise arrays and strings; Category: Coding; Status: Pending	Task row saved; Pending count increases by 1.
Task completion	Change task status from Pending to Completed	Completed count increases by 1; Pending count decreases by 1.
Mock test	Name: Aptitude Practice 01; Score: 8; Maximum: 10	Saved score percentage: 80.0%.
Invalid mock test	Name: Aptitude Practice 02; Score: 12; Maximum: 10	Validation error; no invalid row is saved.
Placement entry	Company: Sample Technologies; Role: Trainee Developer; Status: Applied	Placement row appears in the application list.
Dashboard	2 tasks (1 completed), 1 mock test at 80%, 1 application Applied	Task completion: 50%; test average: 80%; application count: 1.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Output Screenshots

The most useful output evidence is a set of real screenshots taken from the running application. Screenshots should demonstrate the core workflow and complement the test results rather than replace them. A login screen shows the entry point; the dashboard demonstrates combined summaries; the preparation tracker shows task states; the mock-test tracker demonstrates score history; and the placement tracker shows the recruitment-status record. Captions should identify the function shown and be consistent with the figure list.

No executed application screenshots were supplied while this report was prepared. The report therefore uses explicit placeholders rather than images that could be mistaken for genuine output. Before submission, the student should run the reference code, create a fictitious demonstration account, add sample tasks and tests, record placement entries, and capture the interface at a readable size. Replace the placeholder text with these images and remove all bracketed instructions.

An output screenshot should be accompanied by an explanation of what it proves. For example, a dashboard image can show a task total, but the dashboard aggregate test is needed to check whether the total is calculated correctly. A task-list image can show that a record is displayed, but the create/read/update/delete tests are needed to confirm each operation. Evidence should be linked to specific requirements rather than included only to increase page count.

Fig. 6.1: Final Preparation tracker output

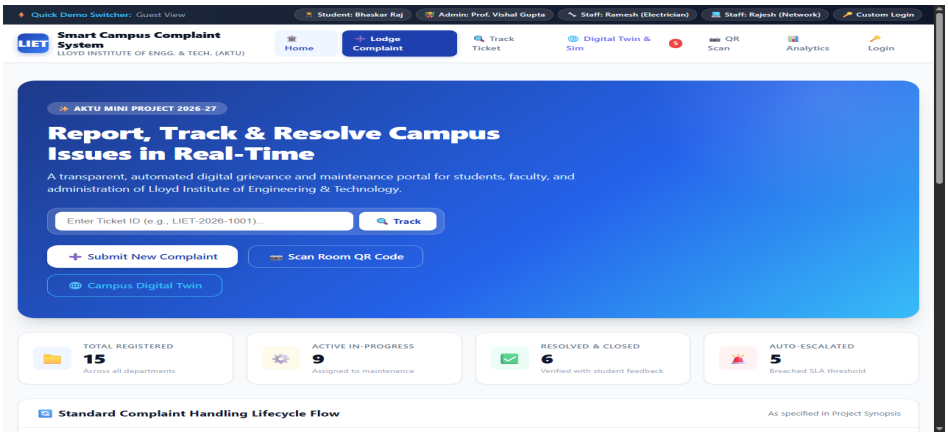


Fig. 6.1 Final dashboard output

Fig. 6.2: Final Preparation tracker output

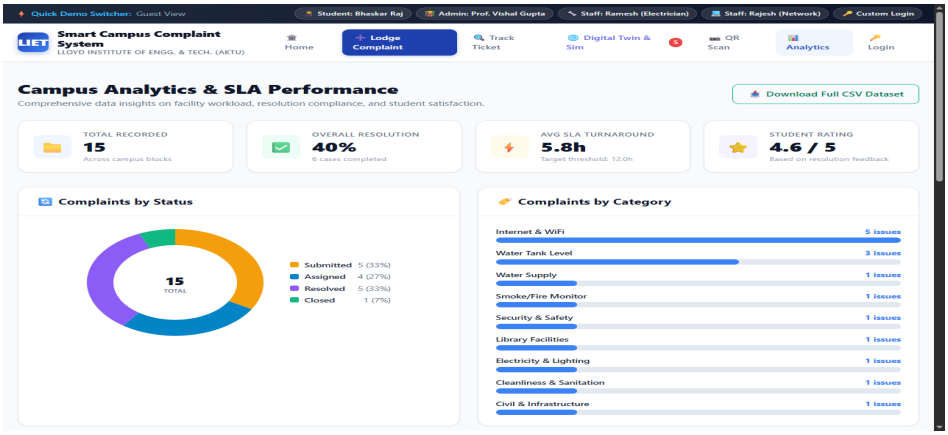


Fig. 6.2 Final preparation tracker output

6.2 Performance / Accuracy Analysis

Performance evaluation should start with the actual application and a defined dataset. Useful measures include the time taken to load a dashboard, the time taken to save a task, and whether the interface remains usable with a representative number of records. Because this mini project is intended for local, individual use, a small timing study can be sufficient, but the environment and method must be recorded. No numerical performance result is stated here because no local runtime measurements were supplied.

Functional correctness can be assessed by comparing known input records with the records retrieved from the database and the values displayed by the dashboard. The test suite should cover empty records, typical records, boundary values, and invalid values. For dashboard counts, create a small dataset with a manually calculated expected total and compare the observed output. For score percentages, use values that have an unambiguous calculation, such as 8 out of 10 giving 80 percent.

Security-related accuracy should be evaluated through negative tests, not inferred from a successful normal workflow. Attempting to open a protected route without logging in checks authentication. Attempting to change another user's record checks record ownership. Submitting quotes or SQL-like content checks whether values are treated as data by parameterised queries. These tests do not constitute a complete security audit, but they provide useful evidence for a second-year prototype.

Usability can be reviewed through a short checklist: can a first-time user locate the main modules, identify required fields, understand status values, and recover from invalid input? If feedback is collected from classmates, record the number of participants, tasks they completed, and the questions asked. Do not claim broad user acceptance based on informal, undocumented impressions. If no participants are involved, present the checklist as a planned review rather than a completed study.

Table 6.1 Evaluation criteria and evidence to collect

Criterion	Measure	Evidence required	Current status
Functional completeness	Count of requirements passing their mapped tests.	Test log with Pass/Fail for FR-01 to FR-14.	Pending execution
Dashboard correctness	Comparison of displayed counts with known records.	Manually calculated expected totals and observed values.	Pending execution
Score calculation	Compare percentages with manually	Use scores with known maximums and	Pending

Criterion	Measure	Evidence required	Current status
	calculated values.	boundary cases.	execution
Response time	Median time for representative page/form requests.	Record machine, browser, dataset size and repeated trials.	Not measured
Access control	Attempt anonymous access and cross-user record changes.	Record request, expected result and observed response.	Pending execution
Usability	Completion of a short user task checklist.	Record observations and participant count if applicable.	Not assessed

6.3 Discussion

The proposed design addresses the identified organisational problem by using a common record model for preparation tasks, test results, and company applications. Instead of asking the student to maintain several independent lists, the system provides distinct modules that share one user identity and one data store. The dashboard offers a concise view of the current state. If the implementation follows the design and passes its mapped tests, the project can demonstrate a practical application of web development and database fundamentals.

The benefits are bounded by manual data entry. The system can show only information that has been entered and updated. It cannot verify whether a drive date is correct, whether a job requirement has changed, or whether a student is ready for an interview. The application should therefore be treated as an organisational aid that complements official employer and institute communications rather than as an authoritative source of placement information.

The security model is a key part of the discussion because the system stores profile and application data. Password hashing and parameterised SQL reduce specific risks, while session checks and per-user queries protect basic account separation. These controls are necessary but not sufficient for a public service. More extensive protection would include CSRF defence for state-changing forms, production secret management, HTTPS, rate limiting, backup controls, logging, dependency management, and security testing.

The choice to exclude recommendations and automated resume screening is appropriate for the initial project. Related research indicates that these capabilities involve modelling and evaluation beyond simple record keeping [1], [2]. The present scope provides a foundation that could support later experimentation if a suitable dataset, clear evaluation method, and privacy safeguards become available. Until then, accurate tracking and a simple, maintainable design offer a more defensible outcome for an individual mini project.

The final evaluation should state both completed functions and remaining work. If the application has been run locally and all relevant tests pass, the report can describe those results with evidence. If some tests remain pending, this limitation should be stated clearly.

Transparent reporting is more useful than a broad statement that the project is “fully successful” without specifying what was built, measured, and verified.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Smart Campus Complaint System is designed to help students organise the everyday activities associated with campus facility maintenance and grievance tracking. The problem addressed is the fragmentation of information across notebooks, spreadsheets, messages, and separate websites. The proposed application combines preparation tasks, mock-test records, company application statuses, and a dashboard in one student-oriented interface.

The system’s first version is based on Python 3, Flask, HTML, CSS, JavaScript, and SQLite. Its modules cover authentication and profile management, preparation tracking, mock-test score entry, company and placement tracking, and dashboard summaries. The architecture separates the presentation, application, and data layers. The logical data model links tasks, tests, and placement entries to a user account. The report also defines functional and non-functional requirements, feasibility considerations, database entities, key algorithms, and test cases.

The project has educational value because it combines several core computing concepts in a focused application: requirements analysis, web routes, form validation, relational database design, CRUD operations, user sessions, password hashing, query parameterisation, data aggregation, testing, and technical documentation. The small initial scope makes the work appropriate for an individual second-year mini project while leaving room for controlled future development.

The report does not claim that the application has been deployed or that runtime tests have passed without supporting evidence. The accompanying reference implementation must be installed and checked locally, screenshot placeholders must be replaced with genuine output, and test results must be recorded before submission. Once these steps are complete, the report can document the verified behaviour of the final build and identify any remaining defects.

7.2 Limitations

The system depends on manual data entry. Incorrect scores, missed updates, or wrong company dates can make the dashboard misleading. It does not independently verify data against official sources. The application should remind users to confirm eligibility criteria, schedules, and recruitment stages using institute or employer communications.

The first version targets a single student’s local workflow rather than a large deployment. SQLite and a small Flask application are appropriate for the intended demonstration but do not automatically provide the operational controls needed by a multi-user public service.

Backups, database migration, monitoring, concurrency handling, and production deployment configuration require additional work if the audience expands.

The dashboard uses simple descriptive measures. It does not infer readiness, predict selection, or compare students. Mock-test percentages can be difficult to compare when tests use different difficulty levels or marking schemes. Completion of a preparation task is a record of activity rather than a direct measure of learning or mastery.

The first version lacks external job-portal integrations, automated resume parsing, job recommendations, reminder services, and dedicated mentor/coordinator accounts. It also requires local verification for usability, browser compatibility, and security. The inclusion of password hashing and user-specific queries reduces selected risks but should not be interpreted as a complete security certification.

Finally, screenshots, test outcomes, and performance measurements are evidence-dependent. Until they have been collected from the final code on the target environment, the relevant entries must remain labelled as placeholders or pending execution. This limitation is a documentation status rather than a claim that the design itself cannot be implemented.

7.3 Future Scope

Reminder support could notify a student about upcoming preparation targets or application deadlines. A reminder feature should use dates entered and verified by the student, provide a way to disable notifications, and avoid sending duplicate messages. If reminders are implemented, the project would need a schedule mechanism and clear rules for missed, updated, or cancelled dates.

Resume skill extraction could be considered after the data model and core functionality are stable. Such a feature would require careful file handling, text extraction, validation of supported formats, privacy controls, and a way for users to correct extracted skills. Automated extraction should assist the user rather than silently overwrite the student’s information.

Job recommendation could be explored as a separate research extension. A responsible prototype would need structured skill profiles and job descriptions, a suitable sample dataset, a chosen matching method, and an evaluation plan. The systematic review by Ertu■rul and Bitirim discusses the range of methods and open issues in job recommender systems [1]. A future student project could begin with transparent rule-based matching before considering machine learning.

Performance charts could display mock-test results over time, category-wise task counts, or the status distribution of applications. The charts should identify the denominator, date range, and calculation method. They should not turn small or inconsistent datasets into misleading

conclusions. If a comparison is added, the report should state how missing values and different test scales are handled.

Mobile-friendly layouts could improve access from phones and tablets. A responsive interface can be developed using CSS breakpoints and semantic HTML without necessarily creating a separate mobile app. Further accessibility work could review keyboard navigation, contrast, labels, focus order, and error messages against relevant accessibility guidance.

A multi-role version could provide a mentor or placement coordinator with limited access to appropriate student information. Such a change would require a role-based access model, explicit permissions, audit records, and agreement about which personal data can be viewed. The student-facing design should not simply expose all records to new roles without a privacy review.

If the project grows beyond a local demonstration, its operational architecture could be extended to a managed database, controlled deployment environment, automated backups, structured logging, and continuous integration. These improvements should be driven by observed needs and a threat model, not added merely for complexity. The guiding principle for any future extension should be to preserve reliable record management and clear data ownership while introducing new functionality in a testable way.

Table 7.1 Limitations and future improvements

Extension	Potential capability	Additional work required
Reminders	Notify about target dates and application deadlines.	Scheduling logic, user preferences, and reliable date handling.
Resume skill extraction	Extract and allow correction of profile skills.	File validation, privacy controls, parser evaluation.
Job recommendations	Match structured student skills to job requirements.	Dataset, transparent method, evaluation, bias review.
Progress charts	Plot test trends and category/status summaries.	Correct aggregation rules and clear chart labels.
Mobile support	Responsive layout for small screens.	Browser and accessibility testing.
Mentor/coordinator roles	Limited access for authorised staff.	Role-based access control and privacy review.
Production deployment	Managed hosting, backups, logging, stronger operations.	Production server, secure secrets, monitoring and incident plan.

REFERENCES

[1] D. Ç. Ertu■rul and S. Bitirim, “Job recommender systems: a systematic literature review, applications, open issues, and challenges,” Journal of Big Data, vol. 12, Art. no. 140, 2025, doi: 10.1186/s40537-025-01173-y.

[2] M. Hamid, F. Hajjej, A. S. Alluhaidan, et al., “Automated resume screening using machine learning: an ensemble-based approach for efficient candidate selection,” Scientific Reports, 2026, doi: 10.1038/s41598-026-54959-2.

[3] Pallets Projects, “Quickstart — Flask Documentation.” [Online]. Available: https://flask.palletsprojects.com/en/stable/quickstart/. Accessed: Oct. 28, 2026.

[4] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: Oct. 28, 2026.

[5] SQLite, “SQL As Understood by SQLite.” [Online]. Available: https://sqlite.org/lang.html. Accessed: Oct. 28, 2026.

[6] OWASP Foundation, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Accessed: Oct. 28, 2026.

[7] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: https://docs.python.org/3/. Accessed: Oct. 28, 2026.

Note: The first two research items in the supplied synopsis were independently checked against publisher or repository records. All technical and research references used in the final submitted report should be rechecked by the student at the time of submission, and every reference retained here should correspond to a claim actually cited in the text.

APPENDIX

Appendix A: Source Code (Submitted Separately, If Required)

In accordance with the instruction that the report must not contain source code, program listings, setup commands, and file-by-file code listings are intentionally excluded from this report. If the department requires source code, it should be submitted separately as a ZIP archive or repository link. This appendix is retained only to document that separation; it does not reproduce code.

Appendix B: Additional Material

The following material should be added by the student as part of final packaging where required by the department: (a) actual application screenshots, (b) a test log with pass/fail observations, (c) the plagiarism/similarity report produced by the approved institutional tool, (d) the source-code ZIP or version-control link, and (e) any additional diagrams or code explanations requested by the guide.

Use fictitious data in screenshots and demonstrations. Avoid attaching the database file if it contains personal data. Ensure that the final PDF and editable Word file have matching page order, chapter numbering, references, and captions. The student should verify whether the institute requires any additional declaration or a specific signed certificate before printing the spiral-bound copy.